UNIVERSITY OF TEXAS ARLINGTON | COLLEGE OF BUSINESS

GAIADHI

2252-INSY-5379-001

BUSINESS ANALYTICS CAPSTONE

PROJECT REPORT

Holos Visualization Enhancements

Leveraging AI/ML for Sustainable Agricultural Insights

Professor: Dr. Jayarajan Samuel

Team 9

Ayesha Anwer
Sonal Sunil Mate
Iqra Amreen
Tejesh Nagender
Arian Riyahivafa

Table of Contents

1. Executive Summary 3
2. Introduction 4
   2.1. Background 4
   2.2. Challenges in Visualization 4
   2.3. Project Objectives 5
3. Project Scope 5
   3.1. Key Areas of Focus 5
4. Project Objectives 6
5. Requirements 6
   5.1. Functional Requirements 6
   5.2. Technical Requirements 7
   5.3. User-Experience Requirements 8
6. Methodology Overview 8
   6.1. Phase 1: Data Preparation 8
   6.2. Phase 2: GEE – Colab – Setup 8
   6.3. Phase 3: Visualization Development 9
7. Implementation Approach 10
8. Code Overview: 12
9. Results and Insights 15
10. Achievements 16
11. Risks and Mitigation 18
12. Conclusion 19
13. Recommendations 19
14. References 21

1. Executive Summary

The Holos Visualization Enhancements project advances the mission of Holos—a cutting-edge platform that leverages crop science and AI/ML technologies to support sustainable, data-driven agriculture. Holos empowers stakeholders—ranging from farmers to policymakers—to evaluate agricultural practices based on environmental and economic outcomes. The following project builds directly on this foundation by addressing persistent limitations in how agricultural data is visualized, accessed, and interpreted.

As the agricultural sector continues to embrace data-driven strategies to confront challenges like food security, climate change, and resource efficiency, there remains a gap in visualization tools that offer the precision, adaptability, and usability required by modern agricultural systems. This project focuses on enhancing the visualization of key agricultural performance metrics across districts and over time, specifically targeting three critical variables: Yield (kg/ha), Water Productivity (kg/ha/mm), and Nitrogen Productivity (kg/kg). These indicators are essential for understanding how efficiently land, water, and nutrients are being utilized in crop production. These advancements bridge the limitations of existing tools and support more informed, data-backed decision-making for stakeholders across the agricultural landscape.

In the existing iteration of Holos Visualization, basic Python libraries were used to generate static district-level maps that offered a snapshot of regional performance. While these visualizations were informative, they lacked temporal interactivity, consistent geographic resolution, and the flexibility needed for comparative analysis. The core problem this project seeks to solve is the inaccessibility of high-quality, dynamic visual tools that allow stakeholders to monitor spatial and temporal variability in agricultural productivity—a key need in the face of growing challenges around food security, climate change, and resource optimization.

To address this, the current phase introduces major improvements through the integration of Google Earth Engine (GEE) and custom HTML/CSS interactivity within the platform. These enhancements allow for high-resolution, cloud-based geospatial rendering of agricultural metrics across multiple years, enabling users to interactively select parameters, explore temporal trends, and visualize spatial patterns at the district level. The newly added interface allows for seamless navigation through time-series data, offering a more intuitive, dynamic, and analytically powerful experience.

Our interface allows stakeholders—including farmers, researchers, and policymakers—to interactively select parameters, adjust layer opacity, and visualize changes over time. This not only improves usability but also significantly enhances analytical depth.

2. Introduction

2.1. Background

Agriculture remains the cornerstone of global food security and economic resilience, particularly in developing regions where livelihoods heavily depend on farming. As the global population continues to rise, the pressure to enhance agricultural efficiency, sustainability, and resilience has never been greater. The sector faces complex challenges—including climate variability, resource scarcity, and the need for environmentally sustainable practices—that demand a strategic fusion of scientific research, technological innovation, and data-driven decision-making.

To address these needs, the Holos platform was developed as a comprehensive decision-support tool that integrates crop science with artificial intelligence and machine learning (AI/ML). However, while Holos offers strong analytical capabilities, its previous visualization system lacked the interactivity, temporal depth, and spatial precision needed for stakeholders to fully interpret complex agricultural data. This limitation significantly reduced the platform's accessibility and practical impact—highlighting the need for improved, user-centered visualization tools capable of delivering dynamic, actionable insights at scale.

2.2. Challenges in Visualization

1. Static Outputs - The original visualizations were entirely static, offering no interactive functionality for users. Each map represented a fixed output, which meant users could not engage with the data dynamically—such as by zooming in, filtering by variable, or adjusting display settings. Furthermore, the system only allowed for viewing one parameter and one year at a time, making it difficult to perform comparative analyses or identify broader patterns across variables or time.
2. Lack of Temporal Analysis - One of the critical limitations of the earlier version was the absence of temporal functionality. Users could not explore how agricultural metrics evolved over time, as there was no built-in support for time series visualization. This lack of temporal depth restricted the ability to identify trends, seasonal variability, or long-term shifts in productivity and input efficiency—key insights necessary for evidence-based agricultural planning.
3. No Interactive Controls - The earlier implementation offered no options for user customization or interaction. Variables such as yield, water productivity, and nitrogen efficiency were hard-coded, and users lacked the ability to select parameters, adjust map opacity, or filter by value ranges. This rigidity reduced the tool's usefulness in real-world decision-making contexts, where flexibility and adaptability to user needs are essential.
4. Single-Variable Display - Another notable constraint was the inability to visualize or compare multiple variables simultaneously. Users were limited to examining one parameter at a time, with no feature for layering metrics or assessing interdependencies between different forms of productivity. This siloed approach hindered holistic analysis and limited the tool's potential for integrated assessments of agricultural performance.

2.3. Project Objectives

The Holos Visualization Enhancements project was initiated to improve how agricultural data is interpreted and utilized for decision-making. The project focused on achieving the following objectives:

- Accurate, district-level visualizations that enable stakeholders to analyze spatial variability in agricultural productivity and resource use.
- Temporal analysis tools that allow users to observe trends across multiple years, supporting long-term planning and policy evaluation.
- Composite visualizations that integrate multiple variables—such as yield, water productivity, and nitrogen efficiency—for a more comprehensive understanding of agricultural performance.
- Interactive and user-friendly interfaces that are accessible to both technical users and non-technical stakeholders, enhancing the overall usability of the platform.

By meeting these objectives, the project strengthens the Holos platform's ability to support sustainable agriculture through precise, scalable, and actionable insights.

3. Project Scope

The Holos Visualization Enhancements project aims to strengthen the decision-support capabilities of the Holos platform by advancing its geospatial data visualization framework. The scope of this project encompasses the development, integration, and refinement of tools that transform static, single-variable maps into an interactive, multi-variable, and time-aware visualization system. This iteration addresses prior limitations—such as the absence of dynamic controls, limited spatial resolution, and lack of temporal analysis—while aligning with Holos's overarching mission of enabling sustainable, data-driven agriculture.

3.1. Key Areas of Focus

- **District-Level Visualization of Agronomic Metrics -** The platform visualizes agricultural performance indicators—specifically yield, water productivity, and nitrogen productivity—at the district level using spatially anchored data points.
- **Time-Series Data Integration** - A major enhancement in this project is the ability to explore trends over time. Users can now visualize how key metrics evolve across multiple years, offering context for understanding seasonal variability, productivity shifts, and the longitudinal impact of interventions.
- **Multi-Variable and Composite Overlays -** To promote holistic agricultural analysis, the platform supports composite visualizations that allow users to explore multiple variables simultaneously. This enables a richer understanding of the relationships between input

use and crop productivity, helping stakeholders identify synergies or trade-offs across variables.

- **Interactive User Interface with Dynamic Controls -** Built using HTML, CSS, and ipywidgets, the interface features dropdown menus for parameter selection, opacity sliders, and a custom time-bar with play/pause functionality. These components make the tool highly usable for both technical and non-technical audiences, greatly enhancing accessibility and user engagement.
- **Integration with Google Earth Engine and Open-Source Tools -** The project utilizes Google Earth Engine for high-resolution spatial rendering and geospatial computation, combined with open-source Python libraries such as geemap, geopy, and shapely. This ensures the solution remains scalable and adaptable for future use cases, including real-time data integration or expanded geographic coverage.

By focusing on these areas, the project delivers a powerful, interactive, and analytically robust tool that significantly enhances the Holos platform's capacity to inform sustainable agricultural practices and policy decisions.

4. Project Objectives

1. Develop district-level visualizations to highlight variation in key agricultural metrics such as yield, water productivity, and nitrogen productivity.
2. Enable time-series analysis by integrating multi-year data, allowing users to observe temporal trends and changes in agricultural performance over time.
3. Create composite visualizations that combine multiple variables to offer a holistic view of resource efficiency and crop outcomes.
4. Design and implement an interactive user interface using HTML, CSS, and Python widgets to improve usability for stakeholders.
5. Integrate Google Earth Engine (GEE) to leverage cloud-based geospatial processing and satellite imagery for high-resolution, scalable mapping.
6. Improve data accessibility and decision-making by allowing users to dynamically select parameters, adjust visualization opacity, and explore temporal patterns with playback features.

5. Requirements

The success of the current phase of the Holos Visualization Enhancements project depends on the integration of a well-defined set of requirements that directly address the limitations of the previous system. These requirements are organized into three main categories: Functional**,** Technical, and User-Experience, each tailored to meet the goals of increasing interactivity, usability, and analytical depth of the Holos platform.

5.1. Functional Requirements

1. **Time-Series Visualization Functionality**
    - The platform must allow users to explore changes in key agricultural metrics across multiple years.
    - This includes interactive year selection and autoplay capabilities for seamless trend analysis.
2. **District-Level Visualization with Spatial Accuracy**
    - The maps must display yield, water productivity, and nitrogen productivity at the district level using accurate geographic coordinates.
    - The system should ensure consistent spatial boundaries that align with official administrative units.
3. **Parameter Selection and Dynamic Filtering**
    - Users must be able to dynamically select variables of interest through a dropdown menu and adjust layer opacity using sliders.
    - This allows for on-demand data exploration without requiring code-level changes.
4. **Composite Metric Overlays**
    - The tool should enable side-by-side or overlaid display of multiple agricultural indicators.
    - This feature is essential for assessing relationships between productivity and resource efficiency.

5.2. Technical Requirements

1. **Integration with Google Earth Engine (GEE)**
    - The system must utilize GEE for rendering high-resolution satellite imagery
    - GEE should enable scalable map generation across time and geographic space.
2. **Use of Open-Source Python Libraries**
    - Libraries such as `geemap`, `geopy`, `shapely`, `matplotlib`, and `ipywidgets` must be employed for map styling, geometry creation, and user interface elements.
    - These components must integrate seamlessly to ensure responsiveness and performance.
3. **Efficient Data Preprocessing and Handling**
    - The system must handle large datasets with multi-year records efficiently.
    - Preprocessing steps should clean, format, and normalize input data to maintain accuracy across years and metrics.
4. **Editable Codebase for Parameter Customization**
    - Code should be structured to allow easy editing of parameters and configurations.
    - Key variables and mappings should be defined in centralized, well-documented sections.
    - Future users should be able to modify or add new metrics without altering core functions.
5. **Support for Headless UI Integration**
    - The system must be capable of running without a graphical interface.
    - This includes the ability to script and automate map generation or updates.

5.3. User-Experience Requirements

1. **Interactive Year Navigation Bar**
    - Users must be able to switch between years using a visually styled, scrollable year bar with active highlighting and play/pause control.
    - This should offer a smooth user experience similar to modern web interfaces.
2. **Visual Clarity and Color Encoding**
    - Maps should use intuitive color gradients to represent metric intensity (i.e., red-to-green scale for yield).
    - Legends and scale bars must be included to guide user interpretation.
3. **Accessibility**
    - The platform should be accessible to a wide audience, including non-technical users such as farmers or policy advisors.
4. **Minimal Loading Time and Responsiveness**
    - The system must generate visualizations quickly (within a few seconds of selection) to maintain user engagement.
    - All interface elements should respond to user inputs without lag or delay.

6. Methodology Overview

The project adopted a systematic approach to guide the effective creation and implementation of the visualization tools. This approach was organized into three main phases, each targeting a distinct component of the project's objectives.

6.1. Phase 1: Data Preparation

The success of any visualization effort begins with high-quality data. For the Holos Visualization Enhancements project, the data preparation process included:

1. **Data Cleaning**
    - Detecting and eliminating errors, duplicate entries, and inconsistencies in the dataset. None were found.
2. **Data Standardization**
    - Ensuring consistency across data formats—such as standardizing date formats, geographic coordinates, and numerical values—to maintain compatibility with visualization tools.
3. **Geospatial Data Alignment**
    - Reviewing and aligning geographic elements like district boundaries and sub-block

6.2. Phase 2: GEE – Colab – Setup

The infrastructure was set up in Google Colab to provide a cloud-based, scalable environment for working with satellite data using Google Earth Engine (GEE). The goal was to streamline authentication, data access, and geospatial processing without the need for local installations—making the workflow accessible, modular, and easy to replicate.

1. **Setting Up Google Earth Engine (GEE) in Google Colab**

   • Google Colab was used as the development environment to eliminate the need for local installations and enable cloud-based access.
   • The notebook was connected to the user's Google Drive to seamlessly access input files such as Excel sheets.

2. **Authentication and Initialization**

   • Google Earth Engine (GEE) was authenticated through a secure login process within the notebook.
   • The environment was initialized with project-specific credentials to enable access to Earth Engine datasets and tools.

3. **Data Loading and Preparation**

   • Input data, such as agricultural yield reports, were uploaded to Google Drive and read into Colab using Python libraries.
   • The setup allowed efficient data handling and ensured all required resources were available for visualization.

4. **Modular and Reusable Structure**

   • The Python-based setup (using `ee`, `pandas`, `geemap`) was modular, making it easy for others to replicate or extend the workflow.
   • Users can plug in new variables or geographic regions with minimal changes.

6.3. Phase 3: Visualization Development

The visualizations were built using Python libraries with an emphasis on clarity, interactivity, and scalability. The goal was to create tools that allow users to explore spatial and temporal patterns in agricultural and environmental data with ease.

1. **Map Rendering with Earth Engine and geemap**

- Google Earth Engine (GEE) was used to access and process datasets.
- The `geemap` library enabled rendering of high-resolution, interactive maps directly within Google Colab notebooks, with functionalities such as layer toggling and time-series sliders.

2. **Dynamic Time-Series Visualizations**

- Annual maps were generated and exported as individual images for temporal analysis.
- These images were then compiled into a GIF timelapse using Python's `imageio` and `imgkit`, providing an animated overview of changes over time.

3. **Customized Layer Styling and Legends**

- Maps were styled to clearly differentiate between variables such as Yield, Water Productivity and Nitrogen Productivity using color gradients and opacity settings.
- Legends and titles were embedded directly into the maps to enhance interpretability for end users.

7. Implementation Approach

Frontend Interactivity

- **Satellite Basemap (Esri.WorldImagery)**
  The map uses Esri's World Imagery basemap to provide high-resolution satellite imagery as a visual backdrop. This enhances spatial context and helps users interpret agricultural data in relation to land features, elevation, water bodies, and terrain.

- **Custom Year Slicer with Playback Controls**
  A horizontally scrollable year selector is positioned at the bottom of the map interface. Each button corresponds to a specific year and triggers a callback to update the map with that year's data. A central play/pause toggle enables automatic year-to-year animation using JavaScript's setInterval() function. The selected year is visually highlighted, and the bar scrolls smoothly to keep it in view.

- **Parameter Dropdown (Picklist)**
  A dropdown widget labeled "Parameter" allows users to switch between three key agronomic metrics:

  - Yield (kg/ha)
  - Water Productivity (kg/ha/mm)
  - Nitrogen Productivity (kg/kg)

  Selecting a different parameter instantly updates the map layer and the color scale without reloading the page.

- **Opacity Slider**
  The opacity slider enables users to control the transparency of the overlaid feature layer, ranging from 0.0 (fully transparent) to 1.0 (fully opaque). This enhances map readability by balancing the visibility of data against the satellite basemap.

- **Integrated Colorbar Legend**
  A dynamic colorbar appears on the map to reflect the range of the selected parameter. It uses

a linear orange-to-green gradient to indicate relative value intensity (e.g., low-to-high yield) and automatically updates when the year or parameter changes.

- **Polygon Clipping to Tamil Nadu Boundary**
  Although data points may exist outside the state border, only those within the Tamil Nadu administrative boundary are displayed. All polygons are clipped client-side using Earth Engine's intersection() function, ensuring that visual overlays are geographically constrained and focused on the region of interest.

Backend Processing and Integration

- **Earth Engine Feature Creation and Styling**
  Aggregated data for each year is converted into an ee.FeatureCollection, where each feature polygon is assigned a color based on its corresponding value. The data is clipped to the Tamil Nadu state boundary (retrieved from the FAO GAUL dataset) to ensure geographical relevance. Feature styling is applied using color attributes stored in the metadata.

- **Efficient Layer Management and Updates**
  Before new layers are added, existing visual elements (active layer and colorbar) are removed to prevent stacking and improve performance. The Tamil Nadu boundary is persistently displayed as a black outline for consistent geographic reference.

- **Playback Functionality via JavaScript**
  The animation loop for playback is managed using client-side JavaScript (setInterval()), ensuring non-blocking, responsive transitions during time-lapse playback.

- **Spatial Grid Generation**
  Each data point is spatially expanded into a uniformly sized bounding box of approximately 24.8 km × 24.8 km, generated using geopy for geodesic distance calculations and shapely to construct rectangular polygons. The box size was intentionally optimized to create a seamless grid layout, ensuring that adjacent polygons align perfectly without visual gaps. This provides a consistent spatial resolution across Tamil Nadu, allowing for a clean, contiguous visualization of agricultural data.

- **Temporal-Spatial Aggregation**
  For each selected year, the dataset is filtered and aggregated by geographic coordinates (latitude, longitude) using pandas. Mean values are computed for the selected agronomic parameter, ensuring smooth and representative spatial visualization across multiple data points in the same region.

- **Dynamic Color Scaling**
  Parameter values are normalized and mapped to a perceptually meaningful orange-to-green color gradient using Matplotlib's LinearSegmentedColormap. Each polygon feature is assigned a hex color corresponding to its value, aiding visual differentiation between low and high productivity zones.

- **Feature Styling via Earth Engine Metadata**
  Each Earth Engine feature is styled using metadata attributes (color, width) stored as a style

dictionary. The entire FeatureCollection is then rendered with Earth Engine's style() function, enabling customized visualization directly from the feature metadata.

8. Code Overview:

1. **Google Drive and Earth Engine Setup**

```
from google.colab import drive
drive.mount('/content/drive')
ee.Authenticate()
ee.Initialize(project='tejeshn3')
file_path = '/content/drive/MyDrive/GEE/Modified_Sorghum_BlockReport_1975-1980.xlsx'
df = pd.read_excel(file_path)
```

Initializes access to Google Earth Engine and mounts Google Drive to retrieve the input Excel file. The data is read into a pandas DataFrame for further spatial and temporal processing.

2. **Required Libraries and Styling for UI**

```
import geemap
import pandas as pd
import numpy as np
import math
from shapely.geometry import box
from geopy.distance import geodesic
from geopy import Point
from IPython.display import display, HTML
import matplotlib.colors as mcolors
import ipywidgets as widgets
from google.colab import output

# === STYLING ===
display(HTML("""
<style>
  ... (CSS for year bar, widgets, dropdowns, slider)
</style>
"""))
```

All necessary libraries for geospatial analysis, visualization, and UI interaction are imported. Additionally, custom CSS is injected to style the year selector bar, dropdowns, and sliders in the Google Colab notebook environment for a clean and user-friendly interface.

3. **Dataset Preprocessing and Parameter Mapping**

```
lat_col = "Center Lat"
lon_col = "Center Long"
df['Year'] = pd.to_datetime(df['Date']).dt.year

parameters = {
    "Yield": "Unadjusted Yield (kg/ha))",
    "Water Productivity": "Yield/App Irrigation Prod(kg/ha/mm)",
    "Nitrogen Productivity": "Yield/App Nitrogen Prod(kg/kg)"
}
```

Extracts the Year from the Date column and stores it in a new column to facilitate temporal filtering. The parameters dictionary maps user-friendly parameter names to actual column names in the dataset for dropdown selection.

4. **Bounding Box and Color Scaling Utilities**

```
def create_bounding_box(point, area_m2):
    ...

def get_color(value, min_val, max_val):
    ...
```

The create_bounding_box() function generates square bounding boxes (~24.8 km per side) around each data point to create a uniform spatial grid. The get_color() function normalizes values and maps them to a color gradient for visual encoding of parameter intensity.

5. **Map Initialization and Tamil Nadu Boundary**

```
m = geemap.Map()
m.add_basemap("Esri.WorldImagery")
m.setCenter(77.6332, 11.0371, 7)

tamil_nadu = ee.FeatureCollection("FAO/GAUL/2015/level1").filter(
    ee.Filter.And(
        ee.Filter.eq("ADM0_NAME", "India"),
        ee.Filter.eq("ADM1_NAME", "Tamil Nadu")
    )
)
```

An interactive map is created using geemap, and a high-resolution satellite imagery

basemap (Esri.WorldImagery) is added for geographic context. The setCenter() method centers the map on Tamil Nadu. The state boundary is retrieved from the GAUL dataset and used later to clip visual layers to ensure regional focus.

6. **Year Selection and Playback Logic**

```
def on_year_click(selected_year):
    ...
output.register_callback('select_year', on_year_click)

def build_year_buttons():
    ...
```

Implements user interaction for selecting specific years or playing through them sequentially. The on_year_click() function updates the current year and re-renders the map. build_year_buttons() renders the custom year bar with interactive buttons and playback control using embedded JavaScript.

7. **Dynamic Map Update and Earth Engine Rendering**

```
def update_map(param_name, selected_year):
    ...
```

This core function handles the creation and display of the styled polygons. It:

- Aggregates parameter values by year and coordinates
- Applies a normalized color scale
- Creates bounding boxes and converts them to Earth Engine geometries
- Clips polygons to Tamil Nadu's boundary
- Adds styled FeatureCollection layers to the map
- Dynamically updates the colorbar and removes old layers for clean rendering

8. **Widgets for Parameter and Opacity Control**

```
param_dropdown = widgets.Dropdown(...)
opacity_slider = widgets.FloatSlider(...)

param_dropdown.observe(...)
opacity_slider.observe(...)
```

Provides interactive controls for users to:

- Choose between Yield, Water Productivity, and Nitrogen Productivity

- Adjust the opacity of the overlay

Widget values are linked to live map updates, ensuring immediate visual feedback.

9. **Final Integration and Display Initialization**

```
m.add_widget(param_dropdown)
m.add_widget(opacity_slider)
build_year_buttons()
update_map(param_dropdown.value, current_year)
display(m)
```

Combines all interactive components—parameter dropdown, opacity slider, year bar, and the default map layer—into a unified, interactive map view. The map is rendered with default values and remains responsive to user inputs throughout the session.

9. Results and Insights

The implemented prototype successfully meets the key objectives outlined in the project brief and provides a solid foundation for scalable, interactive visualization of Holos simulation outputs.

- **Visual Representation of Yield Dynamics**

  The platform allows users to visualize three Holos-derived metrics—Yield, Water Productivity, and Nitrogen Productivity—over time and space. Each parameter is overlaid on a high-resolution satellite basemap, providing geographic context and clarity. These overlays help reveal spatial variability in productivity across Tamil Nadu.

- **Interactive Time Navigation (Timelapse Mode)**

  A central feature is the year slicer and integrated playback control, which mimics the timelapse experience offered by Google Earth Engine. This enables researchers and stakeholders to examine temporal trends, such as yield declines or improvements over the years, which could correlate with climate shifts or sowing strategies.

- **Seamless Integration with Real Geography**

  Each data point is spatially expanded into a bounding box and clipped to the Tamil Nadu state boundary, ensuring a clean and region-specific visualization. The bounding box size was optimized to prevent visual gaps and enable tiling across the entire state, yielding a coherent grid for analysis.

- **Customizable Visualization**

  Users can explore the data through an intuitive dropdown for parameters and an opacity slider, allowing them to fine-tune how overlays interact with the satellite background. This

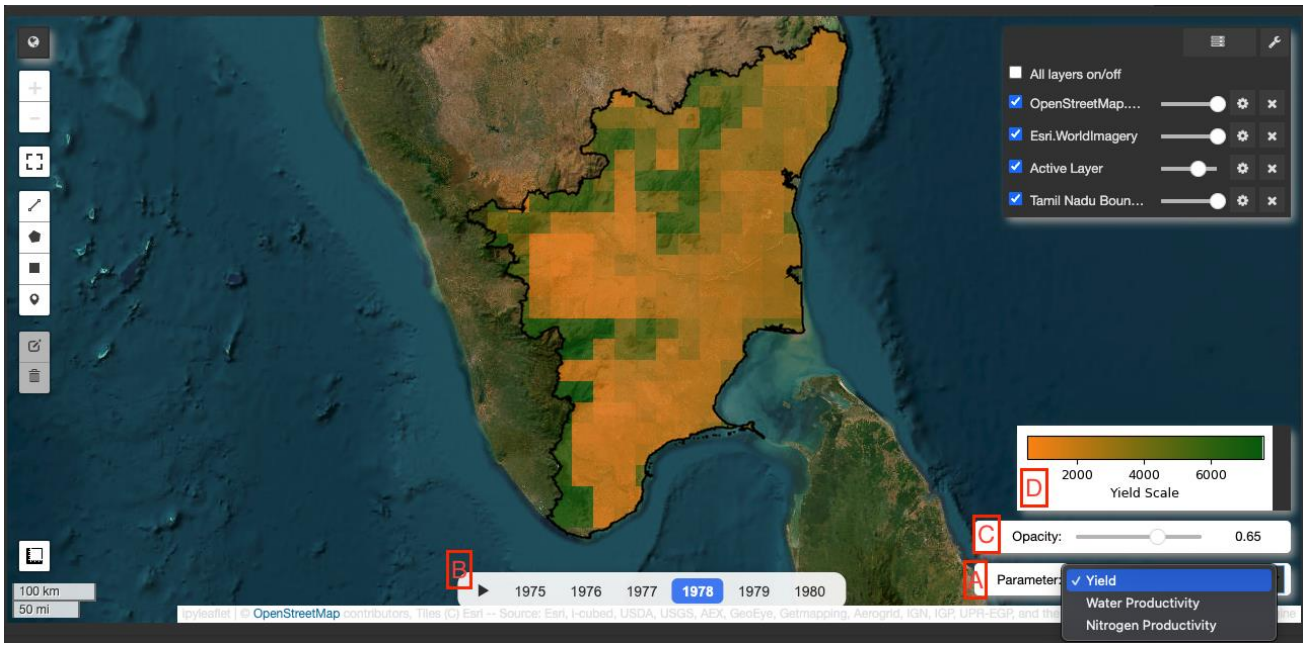
flexibility supports both comparative analysis (e.g., yield vs. water productivity) and visual clarity.

- **Scalable Design Using Open Python Libraries**

  The prototype was implemented entirely using open-source tools such as geemap, ipywidgets, and pandas, closely replicating Earth Engine's interface while maintaining flexibility and avoiding platform lock-in. This aligns with the project's goal of building a system that can be integrated into the Holos web app in the future.

- **Foundation for Climate Impact Evaluation**

  Though current data visualizes raw agronomic parameters, the framework is well-suited to accommodate additional inputs such as climate scenarios, sowing dates, and emission data. This makes it a strong starting point for analyzing the impact of climate change on sorghum yield, as originally envisioned in the Holos simulation framework.



.

Interactive Crop Map – Yield, Water, and Nitrogen Productivity Over Time

A. Parameter picklist allows switching between Yield, Water Productivity, and Nitrogen Productivity.
B. Year slider and play/pause button support temporal navigation.
C. Opacity slider adjusts the visibility of the displayed polygons.
D. The color scale for Yield, Nitrogen, and Water Productivity changes dynamically based on data each year. Future improvements may fix the scale to enhance cross-year comparison.

10. Achievements

Throughout the course of this project, the team was able to meet key technical milestones and

deliver a fully functional prototype that aligns closely with the original goals laid out in the Holos Visualization Enhancements Winter 2025 brief. Below are the major achievements:

Core Functional Deliverables

- Adopted and Implemented the GEE-based Approach
  The project direction was unified around Google Earth Engine, providing a powerful and scalable foundation for spatial-temporal data visualization.
- Integrated Satellite Basemap
  High-resolution satellite imagery (Esri.WorldImagery) was added to provide geographic clarity and visual richness.
- Parameter Selection Interface
  Implemented a clean and intuitive dropdown menu allowing users to toggle between Yield, Water Productivity, and Nitrogen Productivity.
- Year Slicer with Timelapse Playback
  Developed an interactive year selector with integrated play/pause functionality, allowing users to visualize multi-year data as a smooth timelapse.
- Polygons Displayed with Color Encoding
  Data was visualized using geospatial bounding boxes, rendered as styled polygons with a dynamic orange-to-green color scale for better interpretation.

Enhanced Usability and Precision

- Polygon Distance Optimization
  Bounding box sizes were fine-tuned to eliminate gaps between adjacent polygons, ensuring a clean and contiguous grid display.
- Opacity Control Feature Added
  A real-time opacity slider was implemented to let users control the visibility of data overlays against the satellite background.
- State Boundary Overlay and Clipping
  Tamil Nadu's boundary was rendered as a bold black outline, and any polygon data outside the boundary was masked, maintaining geographic focus.
- Resolved Widget Confusion
  Interaction between the dropdown and opacity slider was streamlined, ensuring that each widget updates the map as expected without conflict.

Performance and Polish

- **Reduced Map Flickering and Delay**
  Initial delays (~20s) due to transitions and re-rendering were optimized significantly through backend and UI tweaks.
- **Fully Functional Timelapse Mode**
  A responsive play/pause toggle was added, mimicking Google Earth Engine's timelapse capability and enabling year-to-year animation without blocking UI.

11. Risks and Mitigation

During the course of this project, the team encountered several strategic and technical challenges. However, through adaptive planning and optimization, we were able to mitigate these risks and successfully deliver a functional prototype aligned with Holos' visualization goals.

Learning Curve with GEE APIs

The GEE platform has its own domain-specific API, and building a custom time-aware visualization using both Python (via geemap) and JavaScript required considerable ramp-up, especially for team members new to GEE's client-server model.
Mitigation: We organized internal knowledge-sharing sessions and pair programming sessions to upskill team members on GEE. This collaborative approach helped accelerate development in the later stages of the project.

Dual-Path Development Risk

At the beginning of the project, we explored two parallel approaches:

1. A fully open-source Python-only stack using libraries like geopandas, matplotlib, and folium.
2. A Google Earth Engine (GEE)-centric approach involving Python, JavaScript, and HTML for map rendering and interactivity.

To pursue both directions, our 5-member team was temporarily split into two subgroups, each focusing on one path. However, this bifurcation diluted our efforts and delayed the establishment of a single, unified architecture.

Mitigation: After discussions within the team and alignment with Gaiadhi's interests, we chose to focus entirely on the GEE-based approach, as it offered rich geospatial capabilities and aligned with Holos' interest in scalable, cloud-integrated visualization. This decision helped us streamline development and consolidate our technical efforts in a single direction.

Performance Bottlenecks and UI Lag

When we introduced more complex frontend elements—such as the interactive year slicer and the Tamil Nadu boundary overlay—the visualization latency increased significantly. In some cases, switching years caused a lag of nearly 20 seconds, which was not acceptable for an interactive prototype.

Mitigation: We refactored the code to:
- Optimize data filtering and aggregation logic,
- Reduce redundant redraws and layer reinitializations,
- Clean up previous layers and colorbars before adding new ones,
- Minimize network calls to Earth Engine by caching clipped geometries and using efficient styling.

This effort brought the UI response time down to an acceptable level, ensuring smoother user interactions.

12. Conclusion

This project successfully delivered an interactive, time-aware geospatial visualization prototype aligned with Holos' broader objective of overlaying simulation results on maps in both static and time-lapse formats. By leveraging the power of Google Earth Engine, combined with open source Python libraries like geemap, ipywidgets, and pandas, we were able to create an intuitive interface that enables users to explore agronomic parameters (e.g., Yield, Water Productivity, Nitrogen Productivity) across time and space with a satellite-based geographic context.

While the team initially faced technical and strategic challenges, a unified shift to the GEE approach enabled us to harness advanced geospatial capabilities, including data clipping, dynamic styling, and real-time interactivity. The integration of features such as the year slicer, parameter picker, opacity control, and Tamil Nadu boundary masking provided a robust user experience for domain-specific analysis.

This prototype lays the groundwork for future work in agricultural visualization and climate smart decision-making. With modular code and scalable architecture, it can be extended to support:

- Multiple geographic regions,
- Additional Holos simulation outputs (e.g., emissions, biomass),
- Integration into production-grade web platforms.

Ultimately, this effort demonstrates the value of combining cloud-based geospatial tools with interactive UI elements to bring complex environmental data to life in a way that supports both research and actionable insights.

13. Recommendations

Based on the insights gained from the development and testing of this prototype, we propose the following recommendations to enhance the Holos visualization platform and guide future development:

Pursue Deeper Integration with Earth Engine

Given the capabilities demonstrated through this project, we recommend continuing to explore integration pathways with Google Earth Engine (GEE)—either by embedding Earth Engine Apps or leveraging geemap within a server-backed Python environment like Voila or Panel. GEE's scalability and processing power will be particularly valuable for large-scale simulations and multi-year data.

Incorporate Additional Holos Outputs

The current implementation visualizes three productivity metrics. Future iterations could incorporate more Holos results, such as Emissions data (e.g., N_2O, CH_4), Soil carbon sequestration, Crop

biomass and residue returns. These could be visualized either side-by-side or through parameter toggles.

Performance Optimization for Larger Datasets

As the project scales beyond Tamil Nadu and includes more simulation years or variables, performance could degrade. We recommend:
- Exploring tile-based rendering strategies,
- Leveraging GEE's Image operations for more efficient rendering,
- Introducing asynchronous loading for smoother UI.

Add Download and Export Functionality

Allow users to Download map snapshots (e.g., PNG or PDF) or export filtered data for a selected year and parameter as CSV.

This would enhance the analytical value of the tool for offline reporting and presentations.

Support Offline or Low-Bandwidth Usage Scenarios

For field use or rural applications, consider developing a lite/offline version that allows viewing pre-rendered map tiles or downloading timelapse videos for later use.

Enable Spatial Querying and On-Map Interaction

Improve interactivity by allowing users to:
- Click on a polygon to see detailed values (tooltip or popup)
- Draw or select a region to view aggregated statistics

This would turn the visualization into a lightweight decision-support tool.

Add Scenario Metadata Overlay
Enable the map to display metadata from Holos simulations—e.g., crop type, fertilizer rate, irrigation strategy—via layers or toggle buttons, helping users contextualize what each map represents.

Support Multi-Region Visualization
Scale the prototype to support additional geographies beyond Tamil Nadu. This includes dynamic boundary loading, region selection, and adapting grid sizes based on geography.

Introduce Authentication and Access Control
As the tool matures, consider enabling:
- Role-based access (e.g., admin, researcher, guest)
- Integration with Holos user management

This ensures secure, tailored access to sensitive datasets or premium features.

14. References

Google Earth Engine (GEE)
Official Site: https://earthengine.google.com/
Documentation: https://developers.google.com/earth-engine
Tutorial : G4G 2023 - geemap
geemap: Python Package for Interactive Mapping with GEE
GitHub: https://github.com/giswqs/geemap
Documentation: https://geemap.org/

ipywidgets: Interactive Widgets for Jupyter Notebooks
Documentation: https://ipywidgets.readthedocs.io/

Matplotlib Color Maps
Color Normalization and Custom Colormaps:
https://matplotlib.org/stable/tutorials/colors/colormaps.html

Shapely and geopy for Geospatial Computation
Shapely Documentation: https://shapely.readthedocs.io/
geopy Documentation: https://geopy.readthedocs.io/

Google Colab: Cloud-Based Python Environment
Documentation: https://research.google.com/colaboratory/

Gaiadhi Project Requirements Document
All the material (ppt, pdfs, Git repository, Prior work) provided by: Gaiadhi

Map Style Reference
Esri World Imagery Basemap:
https://www.arcgis.com/home/item.html?id=10df2279f9684e4a9f6a7f08febac2a9